**Islamic University of Gaza**

Faculty of Information Technology

# Software Security Assignment

## Chapter 02: Cryptographic Tools

| | |
|---|---|
| **Student Name** | Mahmoud Basel AL-Juaidi |
| **Student ID** | 120230589 |
| **Course** | Software Security (SDEV 4303) |
| **Chapter** | Chapter 02: Cryptographic Tools |
| **Academic Year** | 2026/2027 (1st Semester) |
| **Teacher** | Dr. Abdelkareem Alashqar |

# A. Questions

### 1. How many keys are required for two people to communicate via a symmetric cipher?

Only **one key**. In symmetric encryption the same key is used to encrypt and decrypt, so the two people share this one secret key. The sender locks the message with it and the receiver opens it with the same key.

### 2. What are the two principal requirements for the secure use of symmetric encryption?

1. **A strong encryption algorithm.** Even if an attacker knows the algorithm and has some ciphertexts (and maybe their plaintexts), he should not be able to find the key or decrypt other messages.
2. **The secret key must be shared and kept in a secure way.** The sender and receiver must get their copies of the key through a safe channel and keep it secret. If someone gets the key, all messages that use it can be read.

### 3. List three approaches to message authentication.

1. **Authentication using symmetric (conventional) encryption:** if only the sender and the receiver know the key, and the message decrypts correctly, the receiver knows it came from the other side (usually with an error-detection code or sequence number inside).
2. **Message Authentication Code (MAC):** a small tag calculated from the message and a secret key, and sent with the message. The receiver calculates it again and compares.
3. **One-way hash function:** a hash of the message is calculated and then protected, for example by encrypting it with a symmetric key, by signing it with a private key (public-key encryption), or by hashing it together with a secret value (like HMAC).

### 4. What are the principal ingredients of a public-key cryptosystem?

- **Plaintext:** the readable message (input).
- **Encryption algorithm:** does the transformations on the plaintext.
- **Public key and private key:** a pair of keys; if one is used to encrypt, the other is used to decrypt.
- **Ciphertext:** the scrambled output, which depends on the plaintext and the key.
- **Decryption algorithm:** takes the ciphertext and the matching key and gives back the plaintext.

### 5. What is the difference between a private key and a secret key?

A **secret key** is used in **symmetric** encryption, and it is **shared** between the two people who communicate (both of them know it). A **private key** is used in **public-key** (asymmetric) encryption, and it is known **only by its owner** and it is never shared with anyone. It has a matching public key that everybody can know.

### 6. What is a public-key certificate?

It is like an ID card for a public key. It contains a user's **public key** plus information about the **owner** (like the name / user ID, and validity period), and all of this is **signed by a trusted third party** called a Certificate Authority (CA). Anyone who trusts the CA can check the signature and be sure that this public key really belongs to that person, so an attacker cannot just publish a fake public key with someone else's name. (Example: X.509 certificates used in HTTPS.)

### 7. How can public-key encryption be used to distribute a secret key?

One common way is the **digital envelope**. If Alice wants to send a secret message to Bob:

1. Alice generates a random one-time symmetric key (session key).
2. She encrypts the message with this symmetric key (fast).
3. She encrypts the symmetric key itself with **Bob's public key**.
4. She sends both: the encrypted message and the encrypted key (this is the "envelope").
5. Bob uses his **private key** to decrypt the symmetric key, then uses it to decrypt the message.

Only Bob has the private key, so only he can open the envelope and get the secret key. Another way is the **Diffie-Hellman** key exchange, where the two sides calculate the same secret key from public values without sending the key itself.

# B. Problems

## Problem 1: Checking the key with XOR

**Yes, there is a big flaw.** The scheme sends two things on the channel that an attacker can see:

- First message (me → partner): $C = R \oplus K$
- Reply (partner → me): $C \oplus K = R \oplus K \oplus K = R$ (the random string in clear!)

An eavesdropper who records both messages just XORs them together:

$$C \oplus R = (R \oplus K) \oplus R = K$$

So he gets the **secret key itself**. The idea "we never sent the key" is wrong, because XOR with a random string is only safe if the random string stays secret, and here the partner sends it back openly.

**Small example (4 bits):** K = 1010, R = 0110. I send C = 0110 ⊕ 1010 = 1100. Partner sends back 1100 ⊕ 1010 = 0110. The attacker sees 1100 and 0110 and calculates 1100 ⊕ 0110 = **1010 = K**.

The attacker does not even need to change anything. Just listening is enough to steal the key, and this is the worst case. A better way is a challenge-response with a hash or MAC. For example, the partner returns $H(K \,||\, R)$ or $MAC_K(R)$, which proves knowledge of K without leaking it.

## Problem 2: Digital Signature (DS) vs. MAC

| Attack | (i) Digital Signature | (ii) MAC |
|---|---|---|
| **a. Message integrity** "Mark" changed to "Oscar" | **Yes, Bob detects it.** Bob verifies auth(x) with Alice's public key. The hash of the changed message does not match the signature. Oscar can't make a new valid signature because he does not have Alice's private key. | **Yes, Bob detects it.** Bob recomputes the MAC of the received message with the shared key and it will not equal the received tag. Oscar can't compute a new MAC because he does not know the secret key. |

| | | |
|---|---|---|
| **b. Replay** same message sent 100 times | **No.** Every copy is the real message with a real signature from Alice, so each one verifies correctly. The signature alone does not tell Bob if he already received it. *Fix:* Alice puts a timestamp, sequence number or nonce inside x before signing, so Bob rejects copies he has already seen. | **No.** Same thing, each copy has a correct MAC. *Fix:* the same idea, a timestamp, sequence number or nonce inside x before computing the MAC. |
| **c. Sender authentication** (Oscar cheats) | **Yes, Bob can clear it.** He checks auth(x) with Alice's public key. If it is valid, it was made with Alice's private key, so Alice sent it. If Oscar really signed it, it would only verify with Oscar's public key. | **Yes, Bob can clear it (for himself).** Only Alice and Bob know the MAC key. Oscar doesn't know it, so he could not have made a valid MAC. Since Bob knows he didn't make it himself, it must be from Alice. |
| **d. Authentication** (Bob cheats) | **Yes, Alice can clear it.** A judge asks Bob to show the message and signature and verifies it with Alice's public key. Bob cannot forge a valid signature without Alice's private key, so if he can't show a valid one, he is lying. This is **non-repudiation**. | **No.** Bob has the same secret key as Alice, so he could have computed the MAC himself. A judge can't know who of them created it. MAC gives **no non-repudiation**. |

## Problem 3: Collision-resistant hash

**No, it is not true.** H maps messages of *any* length into only n bits, so there are only $2^n$ possible hash values, but there are infinitely many messages. By the **pigeonhole principle**, many different messages must share the same hash value, so collisions ($x \neq x'$ with $H(x) = H(x')$) **always exist**.

For example, with SHA-256 (n = 256), just the messages of length 257 bits are $2^{257}$, which is double the number of possible hashes, so at least $2^{256}$ of them must collide.

"Collision resistant" doesn't mean collisions don't exist. It means it is **computationally infeasible to find** such a pair x, x' (it would take too much time, around $2^{n/2}$ tries because of the birthday attack).

# C. Programming Task: RSA Digital Signature in Java

**GitHub repository:** https://github.com/mahmoud20820055-crypto/RSA-Digital-Signature

**Tools:** Java, Apache NetBeans IDE 19.

## 1. The idea

A digital signature with RSA works like this: Alice hashes her message and "encrypts" the hash with her **private key**; this result is the signature. Bob "decrypts" the signature with Alice's **public key** and compares it with his own hash of the message. If they are equal, the message is really from Alice and was not changed.

> Keys: $n = p \cdot q$, $\varphi = (p-1)(q-1)$, $e = 65537$, $d = e^{-1} \bmod \varphi$
> Sign: $S = H(M)^d \bmod n$
> Verify: $S^e \bmod n == H(M)$ ?

I wrote two small programs (both in the package `Book`):

- **RSADigitalSignature.java:** I implemented RSA by myself using Java's `BigInteger` class, so every step is visible (primes, n, φ, d, sign, verify).
- **JavaBuiltInSignature.java:** the same task using Java's ready security library (`Signature.getInstance("SHA256withRSA")`), and this is how it is usually done in real projects.

## 2. Main parts of the code

**Key generation:** two random primes, then n, φ and d:

```
BigInteger p = BigInteger.probablePrime(bits / 2, random);
BigInteger q = BigInteger.probablePrime(bits / 2, random);
n = p.multiply(q);
BigInteger phi = p.subtract(BigInteger.ONE).multiply(q.subtract(BigInteger.ONE));
e = new BigInteger("65537");
d = e.modInverse(phi);          // (e * d) mod phi = 1
```

**Hashing:** SHA-256 of the message, turned into a positive number:

```
MessageDigest md = MessageDigest.getInstance("SHA-256");
byte[] hashBytes = md.digest(message.getBytes(StandardCharsets.UTF_8));
return new BigInteger(1, hashBytes);
```

**Sign and verify**:

```
static BigInteger sign(BigInteger hash) {
    return hash.modPow(d, n);                 // uses the PRIVATE key
}
static boolean verify(String message, BigInteger signature) {
    BigInteger hashFromMessage   = hashMessage(message);
    BigInteger hashFromSignature = signature.modPow(e, n);   // PUBLIC key
    return hashFromMessage.equals(hashFromSignature);
}
```

Note: we sign the hash and not the full message because RSA can only work on a number smaller than n, and the hash is short (256 bits) and fixed in size.

## 3. Running the code and explaining the results

I ran both programs in NetBeans (right-click on the file → Run File). The keys are generated randomly, so the numbers change every time the program runs, but the hash of the same message is always the same.

**Screenshot 1 (RSADigitalSignature): key generation, signing and verifying the original message.**

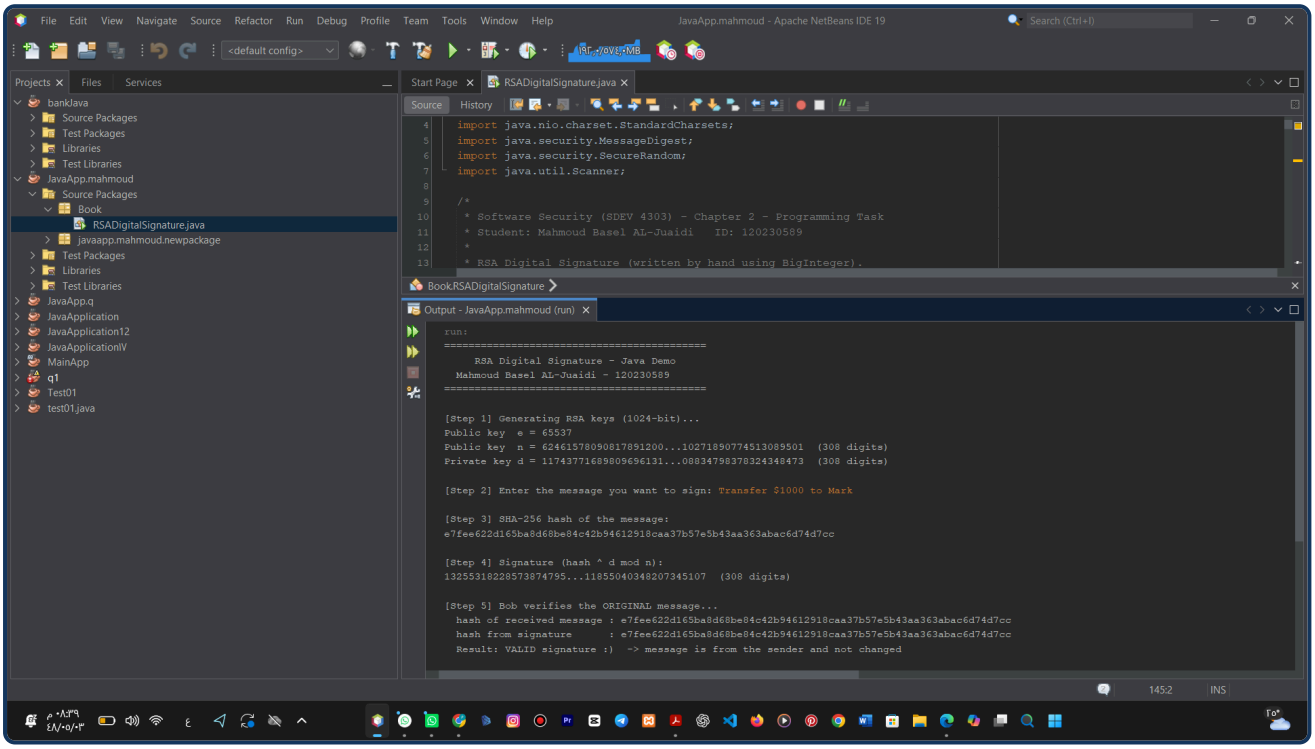



*Figure 1: Output of steps 1 to 5*

- **Step 1:** the program generated a 1024-bit RSA key pair. The public exponent is e = 65537. n and d are very big numbers (308 digits), so I print only the start and the end of them: n = 624615780908178912 00...10271890774513089501 and d = 1174377168980969613 1...08834798378324348473.
- **Step 2:** I typed the message *"Transfer $1000 to Mark"* (the same message from Problem 2).
- **Step 3:** the SHA-256 hash of the message: `e7fee622...6d74d7cc` (64 hex characters = 256 bits).
- **Step 4:** the signature, calculated as $\text{hash}^{d}$ mod n with the private key: 13255318228573874795...11855040348207345107 (308 digits).
- **Step 5:** Bob verifies the message. He hashes the message himself and also computes $\text{signature}^{e}$ mod n using the public key. The two values are exactly the same, so the result is **VALID**.

## Screenshot 2 (RSADigitalSignature): tampering tests.

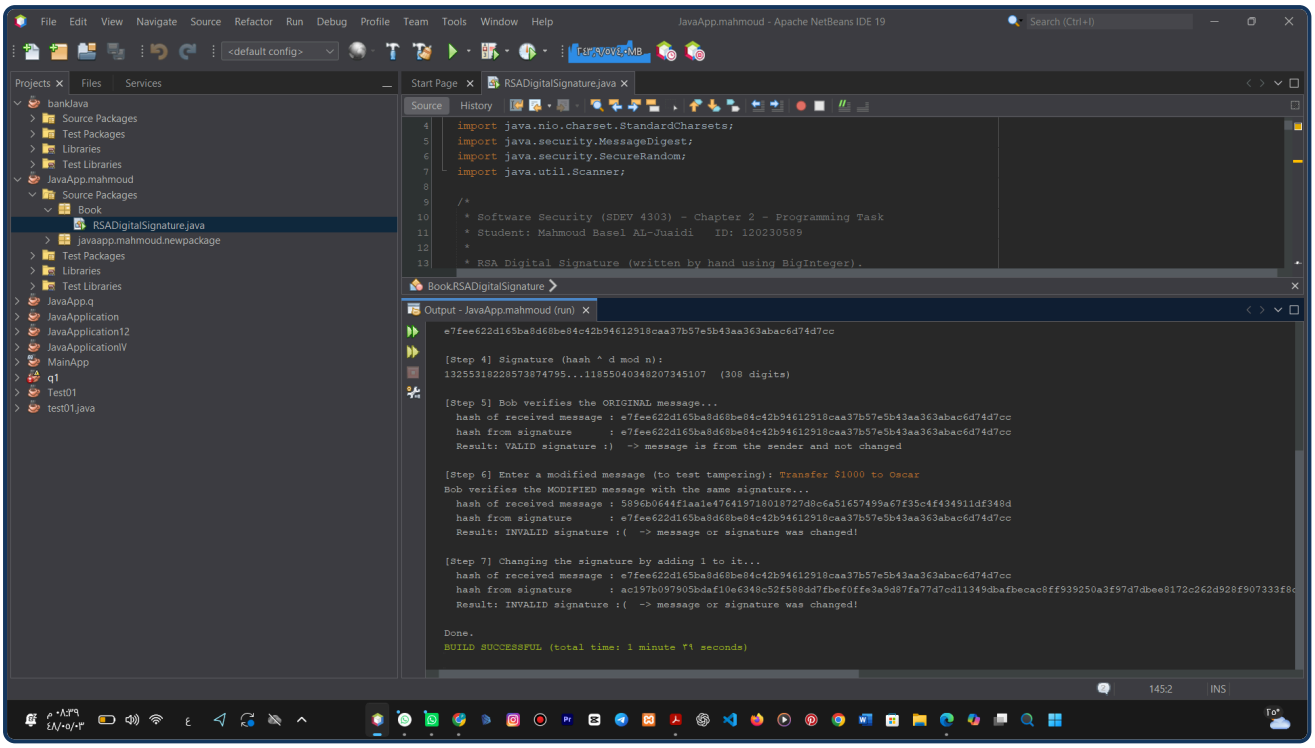



*Figure 2: Output of steps 6 and 7*

- **Step 6:** I acted like Oscar and changed the message to *"Transfer $1000 to Oscar"* but kept the old signature. The hash of the new message (`5896b064...`) is completely different from the hash inside the signature (`e7fee622...`), so the result is **INVALID**. This shows the **integrity** protection from Problem 2(a).
- **Step 7:** I kept the original message but changed the signature just by adding 1 to it. After computing $S^e$ mod n we get a totally random-looking long number (`ac197b09...`), not the hash, so again **INVALID**. This means nobody can edit or fake a signature without the private key.
- At the end NetBeans shows **BUILD SUCCESSFUL**, so the program finished without errors.

**Screenshot 3 (JavaBuiltInSignature): using Java's built-in Signature API.**

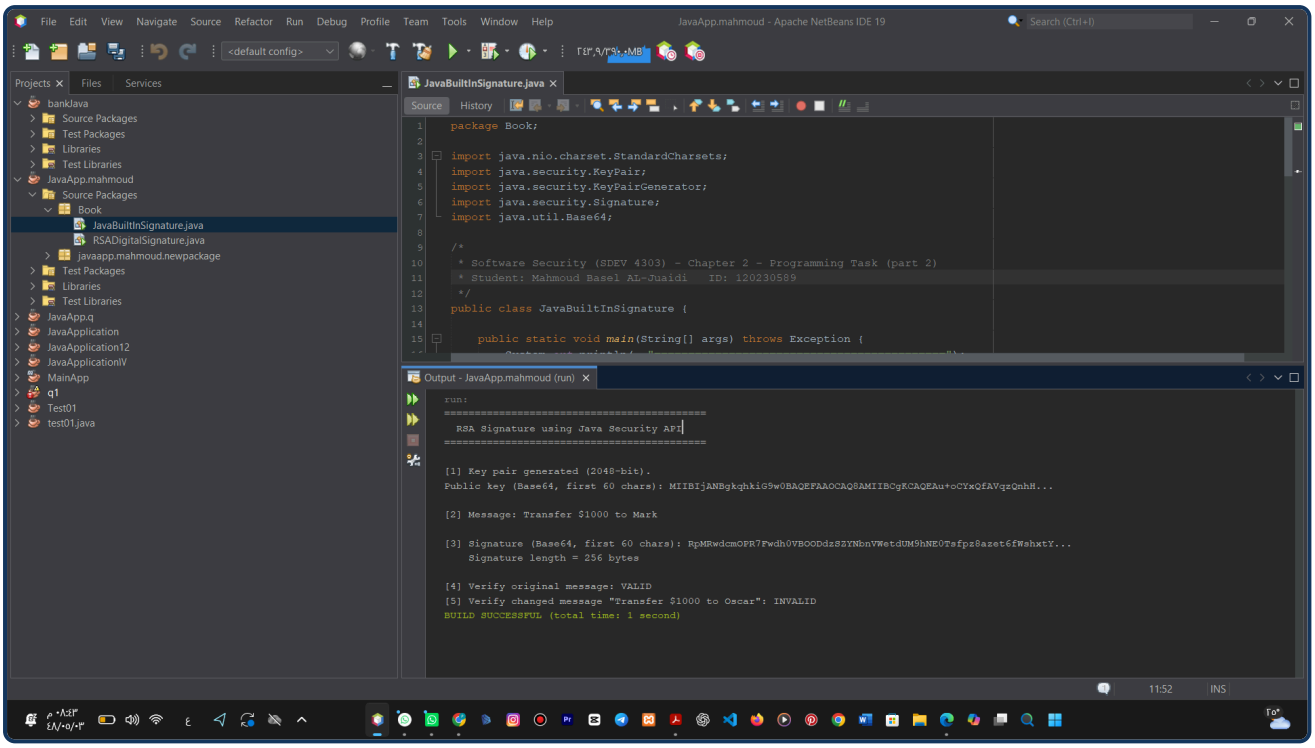



*Figure 3: Output of JavaBuiltInSignature*

- A 2048-bit key pair is generated with `KeyPairGenerator`; the start of the public key is shown in Base64.
- The message "Transfer $1000 to Mark" is signed with `SHA256withRSA` and the private key. The signature is 256 bytes (= 2048 bits, the same size as n).
- Verification of the original message gives **VALID**, and verification of the changed message ("to Oscar") gives **INVALID**. So the library gave the same results as my own code.

## 4. Conclusion

In this task I implemented RSA digital signatures in Java in two ways. From the results I saw that a signature made with the private key can be checked by anyone who has the public key, and that any change to the message or to the signature is detected. Writing the manual version helped me a lot to understand the math behind RSA (n, φ, e, d and modPow). But in real applications it is better to use the built-in version, because it is tested and it also adds padding (PKCS#1) to the hash, which makes it more secure.